VeriEdge: Verification-Aware Orchestration for Trustworthy Decentralized Edge LLM Inference

Anonymous Author(s)

Abstract

Edge devices expose a large aggregate compute pool for AI inference, but turning that pool into a trustworthy decentralized service remains difficult. A practical system must decide how to assemble heterogeneous providers into an execution group, distribute task data without exposing plaintext to the whole network, and verify disputed executions without paying the cost of full deterministic replay on every task. Existing decentralized AI systems show that collaborative inference is feasible, but they leave accountability and verification under-specified; meanwhile, strict checkpoint hashing is brittle under heterogeneous execution, while heavyweight cryptographic verification remains too expensive for routine large-model inference.

This paper presents VeriEdge, a verification-aware orchestration framework for decentralized edge LLM inference. VeriEdge combines lightweight off-chain placement with on-chain commitments and settlement, but treats blockchain as a coordination substrate rather than as the core contribution. The system integrates three components: (1) a lightweight orchestration layer that selects providers for collaborative execution while keeping the decision path deployable; (2) a privacy-preserving task delivery path based on off-chain ciphertext publication and targeted key release; and (3) a layered verification service that uses cheap routine screening and a tolerance-aware sampled tensor chain (TSTC) for dispute escalation under heterogeneous execution.

We implement a prototype on top of EXO, IPFS, and contract-mediated settlement. In our evaluation, the delivery path reduces large-payload task-distribution latency relative to replicated encrypted delivery, and TSTC substantially lowers false positives under honest heterogeneous execution while preserving tamper detection and shard-level localization on the current prefill-focused evaluation object. These results suggest that decentralized edge inference is better framed as a verification-aware systems problem than as a mechanism-centric resource market.

CCS Concepts: • **Computer systems organization** → **Dependable and fault-tolerant systems**; • **Security and privacy** → *Distributed systems security*; • **Software and its engineering** → *Distributed systems organizing principles*; • **Networks** → *Peer-to-peer networks.*

Keywords: edge AI, decentralized inference, heterogeneous systems, verification, orchestration

ACM Reference Format:
Anonymous Author(s). 2026. VeriEdge: Verification-Aware Orchestration for Trustworthy Decentralized Edge LLM Inference. In *Proceedings of Twentieth European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Large language models (LLMs) have made high-quality AI inference a mainstream service, but the underlying compute remains heavily concentrated in centralized clouds [2, 3]. At the same time, a large amount of edge compute remains idle across desktops, laptops, mobile devices, and small servers [6]. Recent decentralized AI systems and prototypes, such as EXO, suggest that these devices can be organized into collaborative inference clusters [4, 5]. The systems question is no longer whether decentralized inference is imaginable, but whether it can be made trustworthy, efficient, and accountable on heterogeneous devices.

Building such a system is harder than simply adding a marketplace on top of idle compute. A requester must reveal enough information for matching and execution, but should not expose the full task to every node in the network. Providers may differ in hardware, numeric precision, and software stack, so honest executions can diverge numerically even when they follow the same shard plan. A verifier that requires full deterministic replay is therefore too brittle for routine use, while a verifier that looks only at final task quality may detect bad outcomes without identifying the responsible node. Finally, any practical deployment must support coordination, settlement, and penalties without putting a trusted centralized operator on the critical path.

EuroSys '27, Istanbul, Turkey

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

We therefore frame the problem as *verification-aware orchestration for decentralized heterogeneous inference.* Matching and settlement still matter, but they are supporting mechanisms in a larger system whose core difficulty is coordinating execution under incomplete trust and hardware heterogeneity.

This paper presents VeriEdge, a decentralized edge inference framework organized around that observation. VeriEdge keeps computation-heavy decisions off the chain, uses the chain for commitments and settlement, and introduces layered verification rather than treating verification as an all-or-nothing property. Routine tasks go through a low-cost screening path based on TIQE-style quality checks [14]. When the system needs stronger accountability, it escalates to a verifier that compares shard-boundary checkpoints. To reduce the false alarms caused by heterogeneous execution, VeriEdge replaces strict tensor hashing with a tolerance-aware sampled tensor chain (TSTC). TSTC preserves first-mismatch localization while relaxing exact equality.

The goal of VeriEdge is not to prove a strategy-proof market or a cryptographically complete verifiable-computation pipeline. Instead, we target a deployable design point for decentralized inference at the edge: cheap enough for everyday orchestration, structured enough for dispute resolution, and explicit about the remaining trust and systems trade-offs.

This paper makes the following contributions:

- We identify *verification-aware orchestration* as the central systems problem in decentralized heterogeneous edge inference, and we refactor the design around that problem rather than around mechanism-theoretic optimality.
- We present VeriEdge, which combines lightweight placement, privacy-preserving task delivery, routine screening, and dispute escalation through TSTC into one accountable execution stack.
- We implement a prototype spanning EXO-based collaborative inference, IPFS-based ciphertext publication, checkpoint capture, and contract-mediated settlement, and we evaluate deployment cost, delivery overhead, and verification behavior on heterogeneous devices.

2 Problem Setting and Design Goals

2.1 System Model

VeriEdge serves inference requesters that submit tasks to a decentralized pool of resource providers. A provider contributes one or more heterogeneous devices and advertises its available capacity, minimum acceptable compensation, and public identity. A task may be mapped to one provider or to a small collaborative group of providers that execute a fixed shard plan. The system includes an off-chain orchestrator, one or more storage backends such as IPFS, on-chain contracts for settlement and disputes, and an external verification service whose reports can trigger slashing or compensation.

2.2 Threat Model

We consider an honest-but-curious network with potentially dishonest providers. Adversarial providers may overstate capabilities, inflate asks, skip assigned computation, tamper with intermediate results, or return incorrect final outputs. Honest providers may still exhibit benign numerical drift because of heterogeneous hardware or kernel stacks. We do not assume a fully trusted centralized coordinator. We also do not claim strong Sybil resistance beyond the economic friction created by stake-backed identities; stronger identity defenses remain future work.

2.3 Design Goals

The design is guided by four goals.

- **Practical orchestration.** The system should support frequent matching and settlement without moving heavy optimization or full execution logic on chain.
- **Selective disclosure.** Only the winning providers for a task should receive the information needed to fetch and decrypt the task payload.
- **Escalable accountability.** Most tasks should use cheap screening, while disputed tasks should have access to a stronger verifier that supports localization.
- **Heterogeneity tolerance.** Verification should distinguish malicious tampering from benign numeric drift across devices whenever possible.

3 System Overview

Figure 1 shows the end-to-end architecture. Requesters and providers register task and resource metadata through contracts that maintain the durable state needed for escrow, settlement, and penalties. An off-chain orchestrator aggregates open tasks and available providers, computes a task placement, and commits the result on chain. The winning providers then receive targeted access to task data, execute collaborative inference, and commit execution artifacts for later settlement and possible audit.

The workflow has five stages. First, requesters publish task metadata and providers register capacity, stake, and identity material. Second, the orchestrator computes a placement and writes the selected allocation on chain. Third, the requester encrypts the task payload, publishes one ciphertext to off-chain storage, and releases access material only to the selected providers. Fourth, the selected providers establish a temporary execution group and run collaborative inference. Fifth, the system either settles directly or escalates to verification when triggered by spot checks or challenges.

This organization deliberately keeps the chain off the latency-critical data path. The chain exists to anchor state transitions that need durable visibility or economic enforcement, not to execute matching or inference logic.

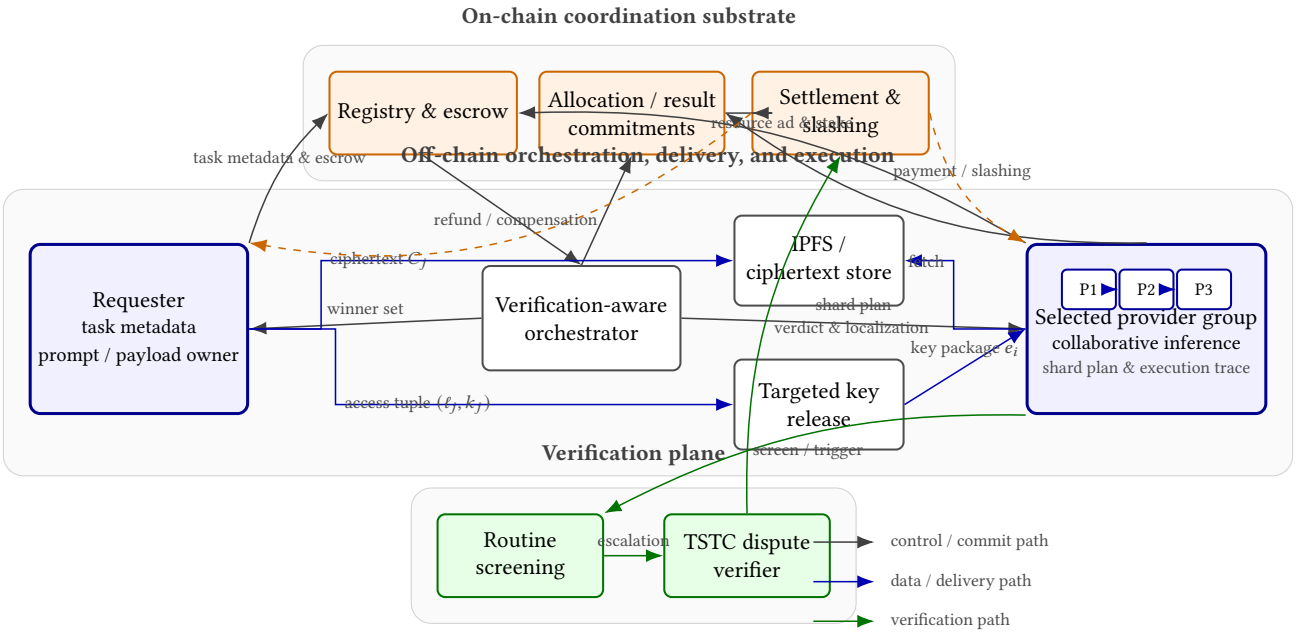

Figure 1. Technical overview of VeriEdge. The architecture separates three planes: an on-chain coordination substrate for escrow, commitments, and settlement; an off-chain plane for placement, selective task delivery, and collaborative inference; and a verification plane that escalates from cheap screening to TSTC-based dispute resolution when accountability is needed.

4 Verification-Aware Orchestration

4.1 Lightweight Placement and Matching

The current prototype uses a lightweight greedy orchestrator. Each provider i reports available capacity c_i, a minimum acceptable compensation a_i, device metadata, and a reputation score $r_i \in [0, 1]$. Each task j reports executable demand d_j and a scheduler priority θ_j. The orchestrator favors high-reputation providers while respecting capacity constraints; ties are broken by lower a_i. Tasks are then placed in descending order of θ_j until they are satisfied or marked failed.

To keep the reputation model executable and tied to system history, we derive r_i from stake, completed-task count, and confirmed fraud count:

$$r_i = \text{clip}\left(w_s \tilde{s}_i + w_t \tilde{t}_i + w_f \tilde{f}_i,\ 0,\ 1\right), \tag{1}$$

where the normalized factors $\tilde{s}_i$, $\tilde{t}_i$, and $\tilde{f}_i$ are computed from on-chain stake, settlement history, and arbitration outcomes. This mechanism is intentionally lightweight and exists only to support the downstream delivery and verification path.

We retain a uniform per-task settlement rule because it simplifies on-chain accounting, but we do not position pricing as the paper's primary novelty. The key systems question is whether this policy exposes the right hooks for network-aware and verification-aware placement. This subsection therefore includes only the machinery needed to connect orchestration decisions to the downstream execution and verification path.

4.2 Privacy-Preserving Task Delivery

Once a placement is finalized, the requester encrypts task payload m_j under a symmetric key k_j,

$$C_j = \mathcal{E}_{\text{sym}}(k_j, m_j), \tag{2}$$

and publishes the ciphertext C_j to off-chain storage such as IPFS. The chain records only a commitment to the plaintext or payload metadata, not the payload itself. For each winning provider i, the requester encrypts the access package (ℓ_j, k_j), where ℓ_j is the storage locator:

$$e_i = \mathcal{E}(\text{pk}_i, (\ell_j, k_j)). \tag{3}$$

Only the selected providers can recover the locator and decryption key.

This design enforces selective disclosure at two layers. First, nodes that do not receive the storage locator ℓ_j cannot directly retrieve the task payload from the storage backend. Second, even if a ciphertext becomes visible, it cannot be decrypted without k_j. This is a lightweight alternative to TEE- or homomorphic-encryption-based approaches and keeps the delivery path compatible with heterogeneous commodity devices.

In the current prototype, we refer to this path as *privacy-preserving delivery* (PPD) and compare it against a baseline called *replicated payload delivery* (RPD), where the requester independently sends a full encrypted payload to each winner. PPD has two practical benefits. First, it avoids replicated ciphertext transmission, which is especially expensive under WAN-like links and large input payloads. Second, it

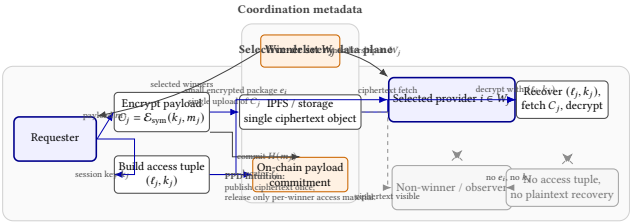

Figure 2. Technical view of privacy-preserving task delivery in VeriEdge. The requester uploads one ciphertext object to shared storage, records only lightweight coordination metadata on chain, and releases a small encrypted access package only to providers selected in the winner set W_j.

aligns access control with the selected execution group: losing providers and unrelated observers cannot fetch plaintext by default.

4.3 Layered Verification

Routine verification and dispute escalation have different cost targets, so VeriEdge separates them instead of forcing one mechanism to do everything.

Routine screening. The first layer uses a TIQE-style two-stage quality check inspired by prior decentralized AI systems [10, 14]. A lightweight semantic filter screens most completed tasks, while a stronger judge model is invoked only for suspicious outputs. This path is intentionally cheap, but it does not provide reliable per-provider attribution.

Dispute escalation. When the system needs stronger evidence, it escalates to a checkpoint-based verifier that inspects shard-boundary tensors. A strict tensor hash chain (THC) provides a deterministic baseline:

$$h_1 = H(T_1), \qquad h_k = H(h_{k-1} \| H(T_k)). \tag{4}$$

Here T_k denotes the checkpoint tensor at shard boundary k. THC localizes the first mismatch, but it is too brittle when honest executions differ slightly across heterogeneous devices.

To reduce false positives, VeriEdge introduces a tolerance-aware sampled tensor chain (TSTC). For each checkpoint T_k, the verifier samples a reproducible coordinate set Ω_k,

$$u_k = S_{\Omega_k}(T_k), \tag{5}$$

quantizes the sampled values under checkpoint-specific tolerance Δ_k,

$$\tilde{u}_k = Q_{\Delta_k}(u_k), \tag{6}$$

and hashes the resulting summary into a chain:

$$c_1 = H(\Omega_1 \| \tilde{u}_1), \qquad c_k = H(c_{k-1} \| \Omega_k \| \tilde{u}_k). \tag{7}$$

This preserves first-mismatch localization while allowing benign drift that stays within a tolerance bin.

In the current prototype, dispute escalation is evaluated in a *prefill-focused* setting. Validators observe only three

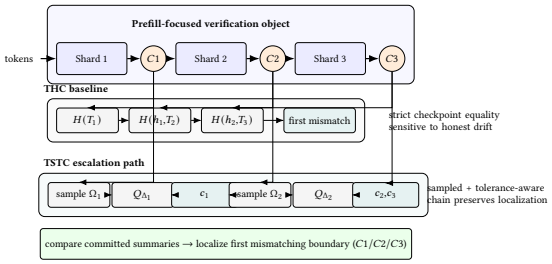

Figure 3. Prefill-focused dispute escalation in VeriEdge. THC applies strict checkpoint hashing at $C1, C2, C3$, whereas TSTC samples coordinates, applies checkpoint-specific tolerance, and chains the resulting summaries to preserve first-mismatch localization under heterogeneous execution.

shard-boundary prefill checkpoints $C1, C2, C3$, and TSTC uses token-channel stratified sampling on these tensors. The tolerance parameter is checkpoint-specific and should be interpreted as calibrated slack for the current heterogeneous stack rather than as a deployment-independent guarantee. Relative to THC, TSTC compares tolerance-aware local stable features rather than requiring exact element-wise equality across devices. The tradeoff is explicit: TSTC reduces false alarms under honest heterogeneity, but it may miss attacks that avoid the sampled coordinates and is therefore not a semantic-correctness proof.

Settlement and penalties. After verification, the chain executes one of four outcomes: normal settlement, provider slashing and requester compensation, failed challenge with fee forfeiture, or verifier-side penalty if the external verification operator misbehaves. This keeps the economic consequence of each execution trace consistent with the system's verification result. The important design point is that dispute escalation is a triggered path rather than an always-on path: validators only pay the cost of checkpoint-based replay when routine screening or an explicit challenge indicates that stronger evidence is needed.

5 Prototype Implementation

We implement VeriEdge as a prototype that combines four components. The orchestration plane runs off chain and consumes task and provider events emitted by the contracts. The data plane uses IPFS for ciphertext publication and EXO-based collaborative inference for multi-provider execution. The chain-facing control plane maintains task registration, provider registration, settlement, and dispute states. The verification plane supports both routine TIQE screening and prefill-focused TSTC-based dispute escalation.

The prototype spans two execution environments. First, the deployment path uses one requester laptop and three provider machines for EXO-based collaborative inference. The requester packages one task, encrypts it, publishes the ciphertext to a local IPFS node, and releases the access package

Table 1. Implementation status of the current VeriEdge prototype.

Plane	Implemented in prototype	Still simplified or deferred
Orchestration	Task/provider registration consumption, lightweight placement, placement commitment	Verification-aware cost model, concurrent-task scheduling, stronger baseline policies
Data delivery	IPFS ciphertext publication, targeted locator-and-key release, commitment anchoring	Larger-scale storage backends, key-rotation policies, winner-group-size scaling study
Execution	EXO-based multi-provider collaborative inference, result collection	Broader workload coverage, larger model families, longer-running deployment campaigns
Verification	Routine screening trigger path, checkpoint capture, THC and TSTC dispute escalation	Always-on exo-native checkpoint export, full-sequence replay, explicit verifier-overhead accounting
Settlement	Contract-mediated task lifecycle and dispute outcomes	Full gas-cost breakdown, richer penalty policy design

only to the selected providers before collaborative execution begins. Second, the dispute-escalation path uses a separate heterogeneous verification harness spanning two Mac mini M4 devices and one Linux RTX3090 server. This harness captures shard-boundary checkpoints, calibrates checkpoint-specific tolerance values, and replays verification traces under mixed numerical precision.

Table 1 summarizes what the prototype already implements and what remains simplified. This split is intentional. The goal is to exercise the interfaces that determine accountability and deployment cost, not to claim that every subsystem is already production-ready.

At the interface level, each task carries a task identifier, model identifier, coarse demand metadata, a commitment to the payload, and the settlement parameters needed by the contract path. Each provider registration carries a public key, device metadata, a compensation floor, and the state needed to recover stake-backed reputation. The orchestrator consumes these events, computes a placement, and writes a compact placement record that names the execution group and the shard plan. The requester then releases one encrypted access package per winner, after which the winning providers fetch the same ciphertext object from IPFS and join one EXO execution group.

The execution and verification traces are also separated intentionally. The normal execution path records output metadata and settlement-relevant state without forcing always-on replay. The escalation path records the information needed to reconstruct a bounded verification object: shard-boundary checkpoint summaries, the challenge context, and the verifier output that determines settlement. This split keeps the common path cheap while still giving the system a way to produce localized evidence when routine screening or a requester challenge demands stronger accountability.

The prototype already supports the key interfaces needed for our claims: task publication, placement commitment, selective task delivery, collaborative execution, result commitment, routine screening, challenge triggering, and dispute escalation. At the same time, several pieces remain intentionally bounded. The placement policy is still lightweight rather than fully verification-aware, and the verifier study focuses on a triggered prefill path rather than full-sequence replay. We therefore present VeriEdge as a practical accountable design point at prototype scale rather than a fully optimized end state.

6 Evaluation

Our evaluation is organized around five questions:

- **Q1:** What is the end-to-end cost of decentralized collaborative inference under LAN and WAN-like conditions?
- **Q2:** How much does private task delivery reduce overhead relative to replicated encrypted delivery?
- **Q3:** Can dispute-escalation verification distinguish tampering from honest heterogeneity?
- **Q4:** What is the cost and sensitivity profile of the dispute-escalation verifier?
- **Q5:** How should orchestration account for network cost and verification risk?

6.1 Methodology and Scope

The evaluation is intentionally path-oriented rather than forcing one aggregate metric to represent the entire system. We evaluate three connected objects: the deployment path, the delivery path, and the dispute-escalation path.

This decomposition matches the system design. The common path is dominated by placement, task delivery, and execution cost, while the exceptional path is dominated by challenge triggering and verification cost. Unless otherwise noted, the deployment and delivery sections use medians over repeated runs, while the verification section uses calibration and evaluation splits that are disjoint at the prompt level.

6.2 End-to-End Deployment Cost

Before turning to runtime cost, we first checked whether changing the EXO deployment scale changes the functional behavior of the model. On a fixed set of 100 GSM8K samples with deterministic decoding, the 1-device, 2-device, and 3-device EXO settings stay within a one-point exact-match range, indicating that deployment-scale changes mostly affect performance rather than broad functional behavior. This gives us confidence that the deployment-cost comparison below is meaningful.

The deployment uses one requester laptop and three Mac mini providers, all running Qwen3 0.6B. For each experimental cell, the requester issues one task containing 20 text questions. The experiment varies the number of provider

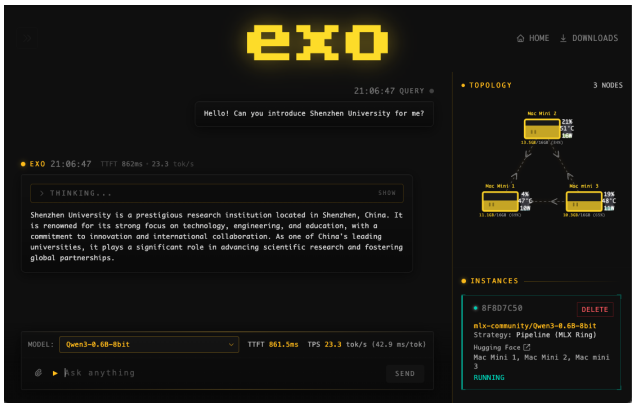

Figure 4. EXO instance abstraction and deployment used in the feasibility evaluation.

Table 2. Main results of the EXO deployment feasibility evaluation under different instance node counts and network conditions.

Nodes	Net	Latency (s)	DL (s)	TTFT (s)	OTPS	Succ. (%)
1	LAN	377.53	0.352	5.254	113.67	99
2	LAN	576.77	0.388	10.339	65.67	94
3	LAN	1015.08	0.683	21.574	13.10	86
3	WAN	1342.53	0.940	32.547	8.07	83

nodes covered by one EXO instance. The LAN setting evaluates $n \in \{1, 2, 3\}$, while the WAN setting evaluates $n = 3$ only. To approximate WAN conditions, we emulate heterogeneous requester-to-provider links with three profiles: Link 1 uses RTT 22 ms, bandwidth 120 Mbps, and loss 0.5%; Link 2 uses RTT 45 ms, bandwidth 60 Mbps, and loss 1.5%; Link 3 uses RTT 80 ms, bandwidth 40 Mbps, and loss 2.0%.

We report five metrics: task latency, download time, time-to-first-token (TTFT), output-token throughput (OTPS), and question success rate. Figure 4 summarizes the deployment abstraction, and Table 2 reports the main results.

The current EXO-based prototype shows a clear monotone degradation as one EXO instance spans more providers. Under LAN, task latency rises from 377.53 s at $n = 1$ to 576.77 s at $n = 2$ and 1015.08 s at $n = 3$, while TTFT increases from 5.254 s to 10.339 s and 21.574 s. OTPS drops from 113.67 tok/s to 65.67 tok/s and 13.10 tok/s over the same settings. Under WAN, the $n = 3$ setting increases further to 1342.53 s task latency and 32.547 s TTFT. This is an important systems result rather than merely a negative one: on the current single-task workload, decentralized inference overhead is dominated by multi-node coordination and shard transfer, not by encrypted task publication.

This observation is important because it shifts the orchestration question from "how do we recruit more providers" to "when is multi-provider inference worth its coordination cost." On the current workload, the dominant overhead comes from collaboration and shard transfer rather than from

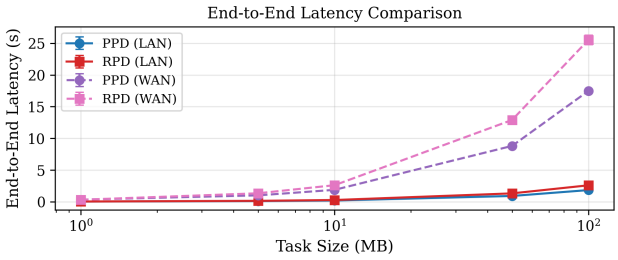

Figure 5. End-to-end latency under PPD and RPD across payload sizes in LAN and WAN.

encrypted task publication. A verification-aware orchestrator should therefore treat group size as a conditional decision that depends on network conditions, expected speedup, and the risk of additional challenge surface.

6.3 Privacy-Preserving Delivery Overhead

We compare two protocols under the same network model. The proposed protocol is PPD, which publishes one ciphertext to decentralized storage, records a commitment on chain, and distributes encrypted (ℓ_j, k_j) only to winners. The baseline is RPD, which sends a full encrypted payload independently to each winner. The experiment uses payload sizes $\{1, 5, 10, 50, 100\}$ MB, fixed winner-set size $|M_j| = 3$, and repeated runs under both LAN and WAN profiles. For task j, end-to-end delivery latency is

$$T_{\text{e2e}} = T_{\text{req}} + \max_{i \in M_j} T_{\text{prov},i}. \tag{8}$$

Figure 5 reports the main result. PPD achieves consistently lower latency at larger payload sizes. At 100 MB, the median latency decreases from 2.668 s to 1.828 s in LAN and from 25.549 s to 17.317 s in WAN, corresponding to 31.5% and 32.2% gains.

The time breakdown explains why the gap widens. Table 3 defines the requester-side components, and Figure 6 shows that PPD is dominated by one-time ciphertext publication, whereas RPD is dominated by replicated ciphertext transmission. For RPD, the dominant transfer term scales approximately with both payload size and winner count. For PPD, the dominant publication term scales primarily with payload size alone, and targeted key delivery operates on a much smaller payload than ciphertext transfer.

This result matters at the systems level because it shows that selective disclosure is not merely a privacy add-on. In the evaluated path, it removes replicated ciphertext transmission from the critical delivery loop and therefore improves both confidentiality and delivery efficiency.

The current study captures only one part of the confidentiality story: it shows that the selected path reduces broad payload exposure by construction and lowers delivery cost on the measured network path. Winner-group-size scaling and key-distribution cost under larger groups remain open,

Table 3. Requester-side time components for PPD and RPD.

Protocol	Symbol	Meaning
PPD	τ_1	symmetric key generation
	τ_2	symmetric encryption
	τ_3	ciphertext upload to IPFS
	τ_4	plaintext commitment hash generation
	τ_5	targeted encrypted key delivery
RPD	τ'_1	key generation
	τ'_2	symmetric encryption
	τ'_3	per-winner key encapsulation
	τ'_4	replicated ciphertext transmission

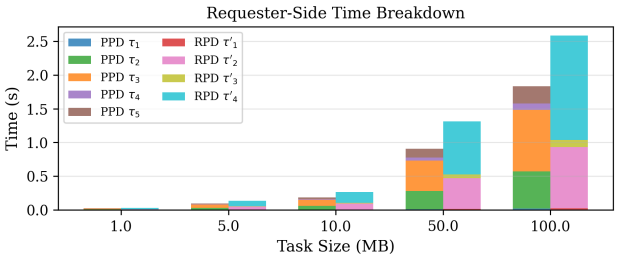

Figure 6. Requester-side time breakdown in LAN for PPD and RPD.

but the current results already justify PPD as the default delivery path in VeriEdge.

6.4 Verification Under Heterogeneous Execution

The dispute-escalation evaluation uses a prefill-focused path with three shard-boundary checkpoints, denoted $C1, C2, C3$. It runs on a heterogeneous execution harness spanning two Mac mini M4 devices and one Linux RTX3090 server. We compare strict THC against TSTC on three scenarios: honest homogeneous execution, honest heterogeneous execution, and tamper. We use disjoint calibration and evaluation prompt splits and report true-positive rate, false-positive rate, and localization accuracy.

The final evaluation considers prefill checkpoints only and uses a fixed small stratified sample corresponding to 16 sampled activations per checkpoint. The tuned tolerance map is $\Delta_{C1} = 0.0022$, $\Delta_{C2} = 0.00525$, and $\Delta_{C3} = 0.02$. The heterogeneous execution profile combines 8-bit Metal on the first shard, BF16 on the second shard, and FP32 on the final shard.

Figure 7 summarizes both the main comparison and the heterogeneity breakdown. On honest homogeneous runs, both verifiers remain clean with FPR = 0. On tamper, both achieve TPR = 1.0 and LocAcc = 1.0 on the current evaluation object. The substantive separation appears on honest heterogeneous runs: THC remains at FPR = 1.0, whereas TSTC reduces the false-positive rate to 0.170111. The right panel shows that this advantage persists across all three heterogeneity levels, with TSTC yielding 0.151333, 0.175333,

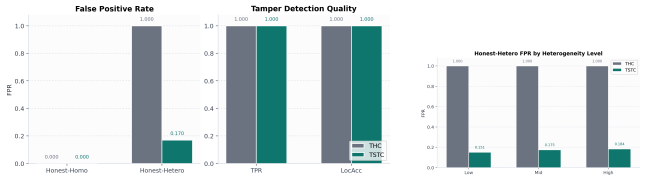

Figure 7. Prefill-focused verifier results on the evaluation split. Left: overall false-positive, tamper-detection, and localization results. Right: honest-heterogeneous false-positive rate across low-, mid-, and high-drift profiles.

and 0.183667 for the low-, mid-, and high-drift profiles, respectively.

Taken together, these results support a restrained but important conclusion: on the current prototype evaluation object, prefill-focused TSTC substantially reduces false alarms relative to strict THC while preserving shard-level tamper detection and localization. This is prototype-scale evidence rather than a deployment-wide guarantee, but it is already strong enough to justify layered verification as a systems design choice.

6.5 Cross-Device Paired Capture

To strengthen the mixed-device claim, the current draft reserves a direct paired-capture study in which matched prompts and shard plans are replayed across concrete hardware/backend pairs. The first row corresponds to the currently available mixed-stack configuration already exercised in the verifier study, while the remaining rows are reserved for ongoing real paired-capture measurements.

Even in this partial state, the evidence chain is already informative. The current mixed-stack row shows that strict checkpoint hashing collapses under heterogeneous execution, while TSTC preserves a usable operating point. Table 4 makes the remaining gap explicit by organizing the still-missing device-pair combinations that will determine how broadly that behavior generalizes.

6.6 Tolerance and Sampling Ablation

We currently characterize one axis of the TSTC operating point through a controlled perturbation sweep. The setting keeps the same three-shard execution boundary and perturbs

Table 4. Cross-device paired-capture result slots. The first row is currently available from the mixed-stack verifier path; the remaining rows are being filled by direct paired-capture runs.

Device / Backend Pair	Scope	THC FPR	TSTC FPR
Current mixed stack (M4/int8, BF16, RTX/FP32)	$C1 - C3$	1.000	0.170
M4/Metal vs. M4/BF16	$C1 - C3$	*pending*	*pending*
M4/Metal vs. RTX/BF16	$C1 - C3$	*pending*	*pending*
M4/BF16 vs. RTX/FP32	$C1 - C3$	*pending*	*pending*

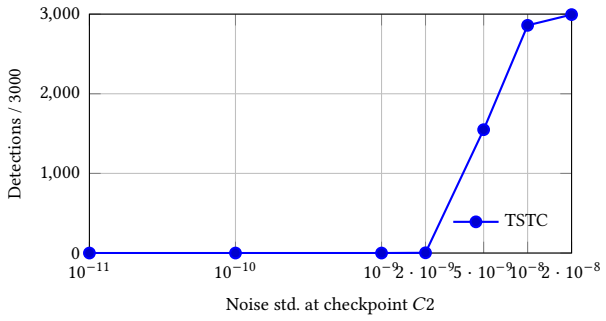

Figure 8. TSTC detection-count response under a prefill $C2$ perturbation sweep.

only checkpoint $C2$ with zero-mean Gaussian noise of normalized standard deviation s. Each sweep point is repeated 3000 times. The broader ablation plan varies sample size and checkpoint-specific tolerance, but the current perturbation study already exposes the most important qualitative behavior: TSTC has a visible tolerance region followed by a sharp detection transition.

Figure 8 shows a clear threshold-like response. From $s = 10^{-11}$ up to 10^{-9}, the detector remains silent. At $s = 2 \times 10^{-9}$, it fires only 3 times out of 3000. The count then rises sharply to 1549 at 5×10^{-9}, 2859 at 10^{-8}, and 2994 at 2×10^{-8}, before approaching saturation. This makes the current calibration interpretable rather than ad hoc. The missing part of the ablation is breadth: sample-size sweeps, checkpoint-specific tolerance comparisons, and real cross-device operating points from Table 4.

6.7 Verifier Operational Overhead

The current verifier path is deliberately scope-bounded: it captures three prefill checkpoints and summarizes a fixed set of sampled activations rather than replaying the whole sequence. At the current operating point, each challenged trace touches

$$N_{\text{sample}} = K \times M = 3 \times 16 = 48 \tag{9}$$

sampled activations in total before chaining them into checkpoint summaries. Table 5 records the current operational scope together with the still-missing end-to-end measurements that will complete the overhead story.

This table is intentionally mixed: the top rows are already established design-scope facts, while the bottom rows mark the measurements that remain necessary for a complete systems evaluation. The important point is that the current verifier is already operationally narrow by design; the open question is no longer what to measure, but only what the final byte and latency numbers are.

Table 5. Current verifier scope and pending operational-overhead measurements.

Metric	Current value	Status
Checkpoint scope per challenge	3 prefill checkpoints	available
Sampled activations per checkpoint	16	available
Total sampled activations per challenge	48	available
Replay scope	prefill only	available
Checkpoint storage footprint	*pending*	E4
Commitment size	*pending*	E4
Verifier replay runtime	*pending*	E4
End-to-end challenge latency	*pending*	E4

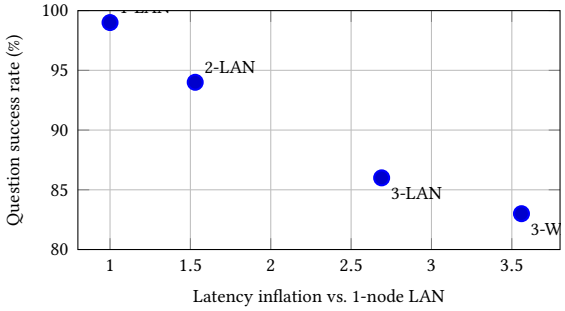

Figure 9. Current placement proxy derived from measured EXO deployment traces. Wider groups move the system toward higher latency inflation and lower success retention.

6.8 Verification-Aware Placement

The current deployment and verifier results jointly motivate a policy study beyond pure verifier accuracy: how should the orchestrator react to heterogeneous execution cost and verification risk? The intended comparison spans random, cost-only, reputation-aware, network-aware, and verification-aware policies. Even before the full head-to-head study is complete, the current deployment trace already provides a concrete proxy result: wider execution groups push the system toward simultaneously worse latency and lower success retention.

Figure 9 makes the proxy result explicit. Relative to the 1-node LAN placement, the 2-node LAN, 3-node LAN, and 3-node WAN settings inflate latency by 1.53×, 2.69×, and 3.56×, respectively, while the corresponding success rates fall from 99% to 94%, 86%, and 83%. On the current workload, this means that a placement policy that ignores network cost and challenge surface will systematically drift toward worse operating points as collaboration width grows.

This proxy result does not replace the full policy study, because it does not yet include explicit challenge-rate or verifier-workload measurements. It does, however, establish the systems direction of E5: the orchestrator should reason jointly about collaboration cost, device mix, and verification risk rather than optimizing only a nominal compensation signal. In separate substrate-level scalability tests inherited

from the original matching prototype, the current greedy off-chain placer remains around 1.4×10^{-1} s at 2000 total participants, suggesting that the barrier to verification-aware placement is not scheduler runtime, but better placement signals and better evidence.

7 Discussion

VeriEdge is built around a simple principle: the chain should carry only the state transitions that benefit from durable visibility and economic enforcement, while orchestration, data transfer, and model execution stay off the latency-critical path. That principle has two implications.

First, blockchain should appear only where it pays for itself: identity binding, commitments, escrow, settlement, and penalty execution. If the chain is not the best place for a function, the system should move that function off chain and say so plainly.

Second, market logic should stay only if it improves a systems outcome that the evaluation can demonstrate. Individual rationality is useful for implementation sanity, but the more important systems question is whether placement should jointly consider network conditions, collaboration risk, and verifier behavior.

Design lessons. Three lessons stand out. First, accountability should be layered: routine screening and challenge-triggered verification serve different cost targets and should not be forced into one mechanism. Second, selective disclosure should be treated as both a privacy and a performance feature, because replicated encrypted delivery can dominate the requester-side critical path. Third, decentralized inference should not equate more providers with better performance: collaboration width has to be justified by actual speedup rather than by resource-pooling intuition alone.

Limitations. The current study still has clear boundaries. The deployment evaluation uses a modest prototype scale and a single workload family. The verifier study is prefill-focused and still includes synthetic perturbation in addition to mixed-device captures. The placement layer is only partially verification-aware. These limitations do not negate the current results, but they do bound the claim strength of the paper. In particular, the verifier claims should remain scoped to the current evaluation object until the real paired-capture, attack-coverage, and overhead studies are complete, and the orchestration claims should remain scoped to the current lightweight placement substrate until policy comparisons are added.

8 Related Work

This paper sits at the intersection of decentralized AI systems, edge inference, and verifiable execution. Prior work on decentralized AI and edge AI motivates the use of idle heterogeneous devices for collaborative inference [5, 11, 13]. Systems such as EXO demonstrate that collaborative inference across commodity devices is feasible, but they do not by themselves answer how such execution should be audited, challenged, or settled under incomplete trust.

Blockchain-backed resource markets provide useful primitives for escrow and settlement [1, 7, 9], but they do not by themselves resolve the systems problem of trustworthy heterogeneous execution. Our design therefore uses the chain as a coordination substrate rather than as a place to execute heavy matching or inference logic.

On the verification side, zero-knowledge systems such as zkLLM and DSperse offer strong long-term cryptographic guarantees [8, 12], but remain too expensive for routine large-model inference. Judge-based screening frameworks help identify suspicious outputs cheaply [10, 14], yet they provide weak attribution. VeriEdge instead targets a layered middle ground: low-cost quality screening for routine tasks and stronger checkpoint-based verification for escalation.

9 Conclusion

Decentralized edge inference is not only a resource-pooling problem but also an orchestration and accountability problem. VeriEdge keeps heavy optimization and model execution off chain, uses the chain for commitments and settlement, and combines selective task delivery with layered verification so that most tasks use cheap screening while disputed tasks escalate to a stronger verifier.

The current prototype already provides encouraging evidence in three places. First, privacy-preserving delivery reduces task-distribution overhead relative to replicated encrypted delivery. Second, the deployment study shows that multi-provider execution introduces nontrivial coordination cost that the orchestrator must reason about explicitly. Third, prefill-focused TSTC reduces false alarms under honest heterogeneous execution while preserving tamper detection and shard-level localization on the current evaluation object. Extending this evidence with real paired captures, overhead accounting, and richer placement comparisons would further strengthen the systems story. Even in its current form, however, the design already shows that decentralized inference can be organized around explicit accountability rather than around blind trust in heterogeneous edge execution.

References

[1] Gaurav Baranwal, Dinesh Kumar, and Deo Prakash Vidyarthi. 2024. Blockchain based resource allocation in cloud and distributed edge computing: A survey. *Computer Communications* 209 (2024), 469–498.

[2] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. 610–623.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020),

1877–1901.

[4] Alex Cheema. 2025. EXO blog. https://blog.exolabs.net/day-1. Accessed: 2025-06-18.

[5] Alex Cheema. 2025. EXO github. https://github.com/exo-explore/exo. Accessed: 2025-06-18.

[6] Shuiguang Deng, Hailiang Zhao, Weijia Fang, Jianwei Yin, Schahram Dustdar, and Albert Y Zomaya. 2020. Edge intelligence: The confluence of edge computing and artificial intelligence. *IEEE Internet of Things Journal* 7, 8 (2020), 7457–7469.

[7] Ummy Habiba, Setareh Maghsudi, and Ekram Hossain. 2024. A Repeated Auction Model for Load-Aware Dynamic Resource Allocation in Multi-Access Edge Computing. *IEEE Transactions on Mobile Computing* 23, 9 (2024), 7801–7817.

[8] Dan Ivanov, Tristan Freiberg, Shirin Shahabi, Jonathan Gold, and Haruna Isah. 2025. DSperse: A Framework for Targeted Verification in Zero-Knowledge Machine Learning. *arXiv preprint arXiv:2508.06972* (2025).

[9] Weidong Li, Jiyi Wu, Liang Zhang, Ke Wang, and Nan Cheng. 2024. Double auction mechanisms in edge computing resource allocation for blockchain networks. *Cluster Computing* 27, 4 (2024), 3017–3035.

[10] Hongbo Liu, Jiannong Cao, Bo Yang, Dongbin Bai, Yinfeng Cao, Xiaoming Shen, Yinan Zhang, Jinwen Liang, Shan Jiang, and Mingjin Zhang. 2025. PolyLink: A Blockchain Based Decentralized Edge AI Platform for LLM Inference. *arXiv preprint* (2025). Available at https://github.com/IMCL-PolyLink/PolyLink.

[11] Tobias Meuser, Lauri Lovén, Monowar Bhuyan, Shishir G Patil, Schahram Dustdar, Atakan Aral, Suzan Bayhan, Christian Becker, Eyal De Lara, Aaron Yi Ding, et al. 2024. Revisiting edge ai: Opportunities and challenges. *IEEE Internet Computing* 28, 4 (2024), 49–59.

[12] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4405–4419.

[13] Zhipeng Wang, Rui Sun, Elizabeth Lui, Vatsal Shah, Xihan Xiong, Jiahao Sun, Davide Crapis, and William Knottenbelt. 2024. SoK: Decentralized AI (DeAI). *arXiv preprint arXiv:2411.17461* (2024).

[14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36.